

```

❸ ballX = ballX + xSpeed
   ballY = ballY + ySpeed

# 9 - Clear the window before drawing it again
window.fill(BLACK)

# 10 - Draw the window elements
window.blit(ballImage, (ballX, ballY))

# 11 - Update the window
pygame.display.update()

# 12 - Slow things down a bit
clock.tick(FRAMES_PER_SECOND)

```

---

*Listing 5-6: A location-based animation, bouncing a ball around the window*

We start by creating and initializing the two variables `xSpeed` and `ySpeed` ❶, which determine how far and in what direction the image should move in each frame. We initialize both variables to the number of pixels to move per frame (3), so the image will start by moving three pixels to the right (the positive x direction) and three pixels down (the positive y direction).

In the key part of the program, we handle the x- and y-coordinates separately ❷. First, we check to see if the x-coordinate of the ball is less than zero, meaning that part of the image is off the left edge, or past the `MAX_WIDTH` pixel and so effectively off the right edge. If either of these is the case, we reverse the sign of the speed in the x direction, meaning it will go in the opposite direction. For example, if the ball was moving to the right and went off the right edge, we would change the value of `xSpeed` from 3 to -3 to cause the ball to start moving to the left, and vice versa.

Then we do a similar check for the y-coordinate to make the ball bounce off the top or bottom edge, as needed.

Finally, we update the position of the ball by adding the `xSpeed` to the `ballX` coordinate and adding the `ySpeed` to the `ballY` coordinate ❸. This positions the ball at a new location on both axes.